

Heuristic Drone Routing for Pheromone Delivery

14 November 2025

Team Collective Solutions

Yana Padte <ypadt001@ucr.edu>

Saumya Chourasia <schou065@ucr.edu>

Anokhee Shah <ashah174@ucr.edu>

Allen Biju <abiju009@ucr.edu>

Parth Awade <pawad006@ucr.edu>

⇒ Github – <https://github.com/saumyaxc/Many-Delivery-Drones>

Abstract

Our client, Mr. Keogh, requested a system to improve the efficiency of a pheromone-delivery drone network used to control the navel orangeworm infestation on his almond farms [1]. The drones disperse pheromone paste to disrupt mating behavior and reduce crop damage sustainably, but the limited flight time makes route optimization essential. In the first phase, we implemented three algorithms (Random Search, Nearest Neighbor, and Augmented Nearest Neighbor) to compute efficient single-drone flight paths. In this phase, we expanded the system to handle multiple drones by integrating K-means clustering with the Nearest Neighbor Algorithm. The clustering algorithm partitions the delivery sites into separate drone regions and determines optimal landing pad locations for up to 4 drones. Within each cluster, the Nearest Neighbor heuristic computes an efficient delivery route. This approach minimizes total travel distance while evenly distributing the workload amongst drones, improving scalability and efficiency across larger orchards.

Elicitation

Meeting with Dr. Keogh: November 12, 2025

Key Questions:

1. Do we calculate/output the total time that each algorithm will take to complete?
2. If 1 drone exceeds 6000 meters, is the expected output a message warning Mr. Keogh that the drone will exceed the distance limit if selected?
3. What is the expected output if the runtime exceeds 5 minutes?

Clarifications Received: All outputs should match the slides exactly (no cost calculations). Any routes that surpass 6000 meters should display a warning message and continue running. Algorithm runtime should not exceed 5 minutes.

Final Understanding: The output after running the algorithm should be printed exactly as shown on the slides, with no time calculation. Whenever the drone exceeds 6000 meters, the program should continue running, but a warning message should be displayed on the screen. Since K-means runs quickly, there should be no circumstance in which the number of coordinates is under 4096 and the run lasts longer than 5 minutes.

Requirement Specification

We begin with Table 1, which lists all acronyms used.

Table 1: Acronym List – Key for the acronyms referenced in this report

CLI	Command Line Interface
BSF	Best-So-Far
NN	Nearest Neighbor
KNN	K-means Nearest Neighbor
I/O	Input/Output
CPU	Central Processing Unit
TXT	Text File
JPEG	Image File
OF	Objective Function
TSP	Traveling Salesman Problem

3.1 System Context Diagram

Figure 1 shows the system context diagram for our algorithm. In the terminal, our Route Optimization CLI reads from farm coordinate files and outputs the corresponding clusters for 1, 2, 3, or 4 drones. It then outputs the best route for each drone and the best total distances. It interacts with the file system for inputs, the console for results, and saves to a directory of solution route files.

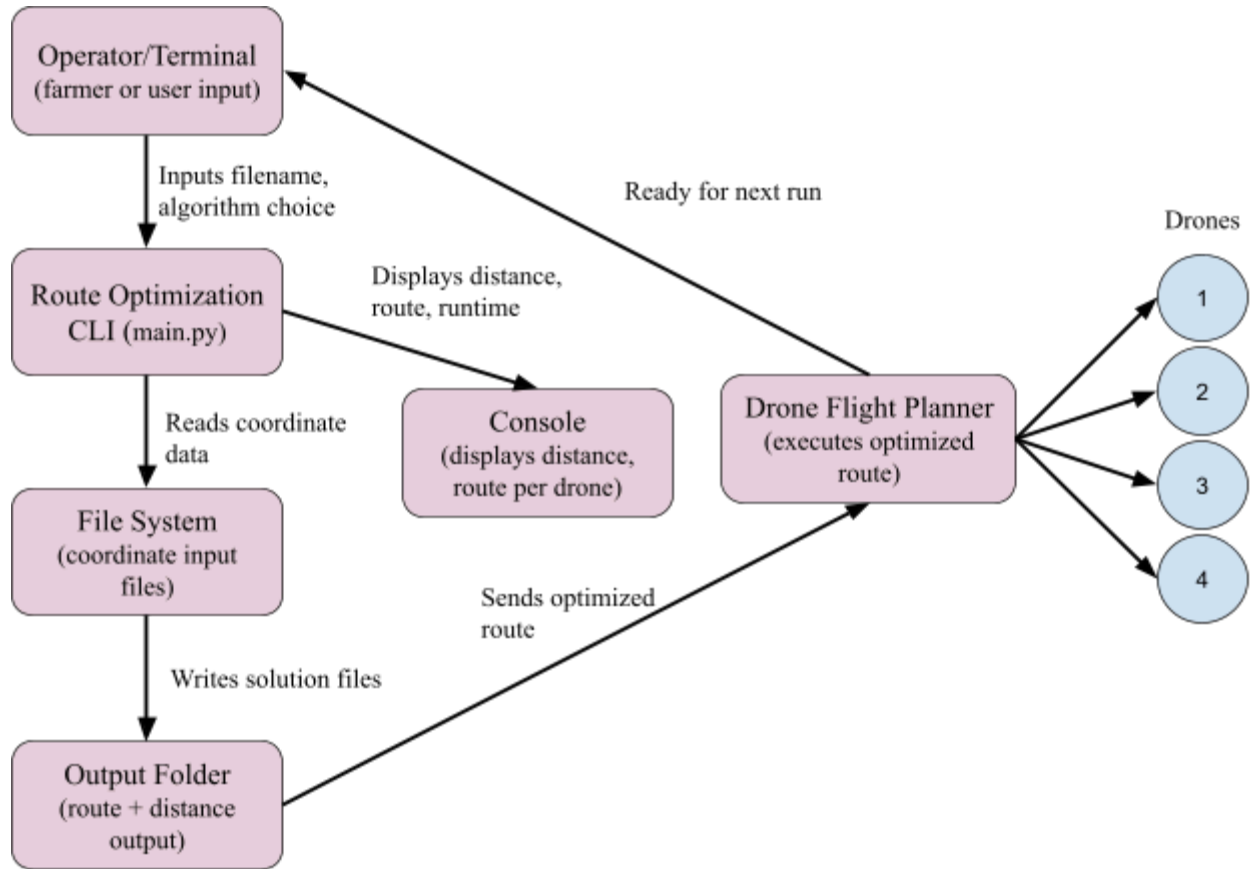


Figure 1: Route Optimization CLI workflow showing cluster generation, drone routes, and total distances.

3.2 System Functions

Figure 2 shows the core workflow of the route-optimization system. The K-means algorithm first processes the coordinate input file to generate cluster assignments for 1–4 drones. Those clusters are then passed to the Nearest Neighbor algorithm, which computes the best route for each drone, along with the total distance and a visual output.

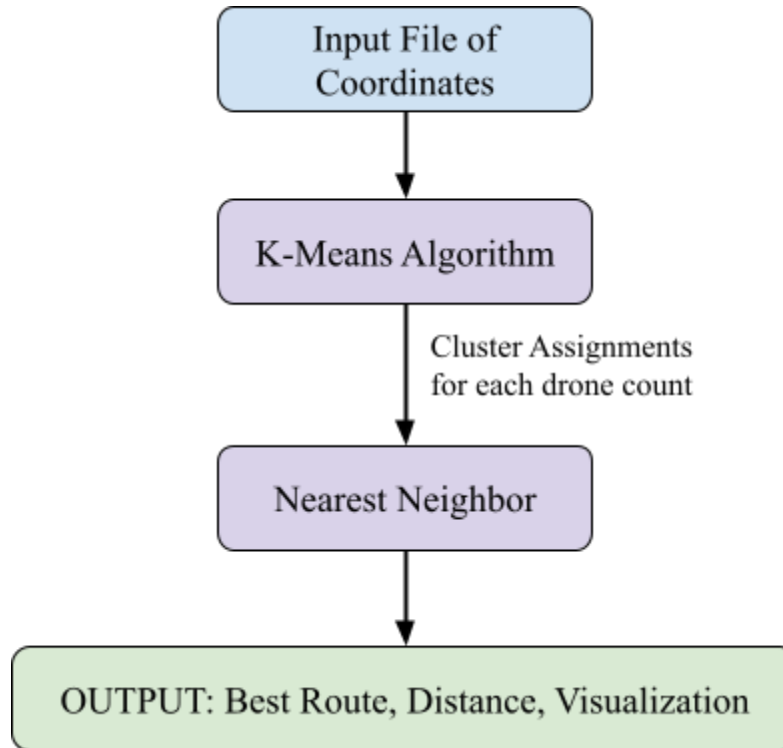


Figure 2: Route-Optimization Workflow

The drone supports the following primary system functions:

1. Verify the input file format containing the complete list of delivery coordinates.
2. Parse the file to populate an array of coordinates and validate that the number of nodes does not exceed 4096 [5].
3. Execute K-means clustering for $K = 1, 2, 3$, and 4 drones to determine optimal landing pad positions. K-means clustering also assigns points to the logical group for each cluster.
4. Assign each coordinate to its nearest landing pad, forming clusters of delivery sites.
5. For each cluster, run the Nearest Neighbor algorithm to compute the delivery route, with the start and end points at the landing pad.
6. Compute total route distance using Euclidean Distance across all drones and output the results of our final algorithm, including landing pad locations, cluster route distances, and total distance.
7. Prompt the user to choose how many drones they would like to use.
8. Save the individual route files, labeled by drone number, from step 7, along with a combined visualization of all clusters, to a separate 'solutions' folder.

3.3 Operating Environment

- Standard laptop using Python
- Local file access only
- Dependencies limited to Python standard libraries, with Matplotlib support for visualization
- Benchmark environment's CPU is Apple M2 Pro (12-core) at up to 3.5 GHz.

3.4 Design & Implementation Constraints

- Maximum of 4096 locations/nodes per input file.
- The system must complete all $K = 1$ to 4 clustering and routing computations within 5 minutes of execution.
- Input and output files use fixed numeric formats. The input file contains (x, y) coordinates, and the output file contains the best route.
- Euclidean distance is used to compute travel distances.
- K-means clustering is repeated 10 times with random initializations to reduce the possibility of finding a poor solution.

3.5 Assumptions and Dependencies

- Input always valid numeric data
- K-means clustering converges before the iteration limit is reached
- Regardless of cluster size, drones start at the calculated landing pad coordinate and return to the landing pad after the route is complete
- Distance metric = Euclidean.
- File I/O is local.

Design

4.1 Workflow Summary

Figure 3 outlines the whole processing pipeline of the multi-drone pheromone-delivery system. The program starts by loading a dataset of coordinates from a text file, where each point represents a delivery location, and computing the corresponding distance matrix. It then applies the K-means clustering algorithm to divide all coordinates into one, two, three, or four regions, with each region corresponding to a drone's service area and the cluster centroid serving as the drone's landing pad. The user selects the number of drones, and the system outputs the respective drone routes in the solutions folder along with an accompanying flight path visualization.

After clustering, the TSP Nearest Neighbor algorithm determines an efficient route for each drone, starting and ending at its landing pad [3]. The algorithm visits every point in the cluster exactly once, always moving to the nearest unvisited location, producing smooth, compact paths. To ensure reliability, K-means is run 10 times for each configuration, and the result with the shortest total route distance is selected. The program outputs the total route length, landing pad locations, and the number of delivery points served by each drone. Each route is saved as a text file, and the system generates a comprehensive visualization showing all drones, their routes, and landing pads on a single plot.

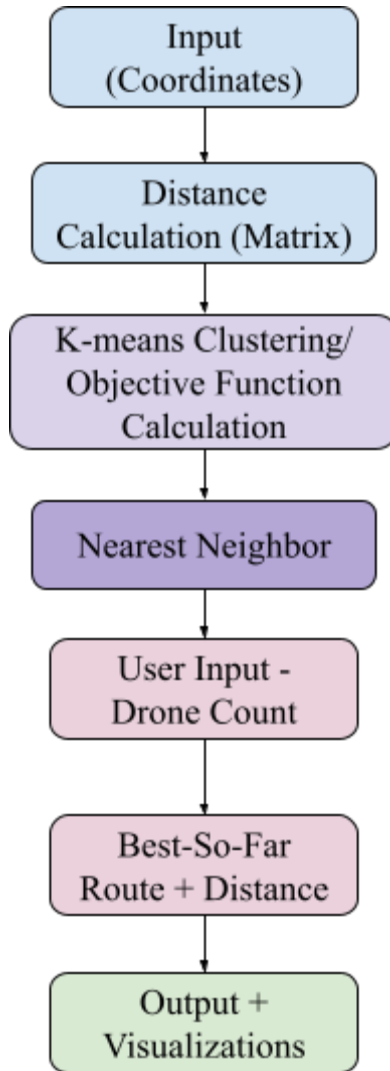


Figure 3: End-to-End Processing Pipeline for the Multi-Drone Delivery System

Insight

The results of our final algorithm, shown in Figures 4 through 18, indicate that K-means clustering effectively partitions the delivery area into localized regions, thereby simplifying and improving routing efficiency. Figures 23 through 25 show that as the number of drones increases, the total travel distance drops sharply from one to two drones but shows more minor improvements beyond that point, forming a clear elbow pattern.

Within each cluster, the Nearest Neighbor algorithm produces routes that follow the data's spatial layout [8]. For circular distributions, the paths naturally trace the perimeter, confirming that combining K-means for partitioning and Nearest Neighbor for local routing produces balanced, efficient solutions. Overall,

this approach achieves strong performance with low computational cost and provides practical routing plans for multi-drone operations [6].

Implementation

Our project was implemented in Python and organized into four main files: *main.py*, *kmeans.py*, *nearestNeighbor.py*, and *visualize.py*. The *main.py* file is the program's entry point. It loads the coordinate dataset from a text file, runs the K-means clustering algorithm for one to four drones, asks the user how many drones to test, and outputs a series of solution files and a visualization for the number of drones input. For each case, it performs 10 runs and keeps the best centroid (launchpad location) based on the smallest objective function value. It then uses these calculated locations to find the best delivery path for each drone.

The *kmeans.py* file contains the main logic for dividing the coordinates into clusters, where each cluster represents one drone's delivery region. Once the clusters are formed, the program calls the *NearestNeighborDistance* function from *nearestNeighbor.py* to calculate an efficient route within each cluster. Each route begins and ends at the drone's landing pad, and the total travel distance is computed by adding up all individual routes.

The *visualize.py* file handles route plotting with Matplotlib. It shows each drone's delivery paths and landing pads in a single figure, separated by color [9]. The visualization helps confirm that the clustering and routing worked as expected and that each drone covers a separate part of the delivery area. Each plot is saved automatically as a JPEG file in the solutions folder, and the program also saves each route as a text file for reference.

Together, these functions create a complete program that clusters delivery locations, generates efficient routes, and produces precise visual results for our multi-drone pheromone-delivery algorithm.

6.1 Naming Convention of Solution Files

→ **Best Route Solution:** /solutions/<filename>_<drone number>_SOLUTION_<ceiling value of best distance>.txt

→ **Visualization of Best Route:** /solutions/<filename>_OVERALL_SOLUTION.jpeg

6.2 Algorithm Traces

Figures 4 through 18 show the CLI outputs for our K-means clustering and Nearest Neighbor algorithms on three datasets, along with the resulting path visualizations. Each case shows the route distances produced for each drone count and includes the route file output.

Case 1 – Almond9832.txt

→ **Inputs:** *Almond9832.txt*, 1

```
● (base) allenbiju@Allens-MacBook-Pro-2 src % python main.py
Enter the name of file: Almond9832.txt
There are 1798 nodes: Solutions will be available by 10:11pm

1) If you use 1 drone(s), the total route will be 8374.3 meters
  1. Landing Pad 1 should be at [50,2], serving 1800 locations, route is 8374.3 meters

2) If you use 2 drone(s), the total route will be 8228.4 meters
  1. Landing Pad 1 should be at [50,80], serving 884 locations, route is 4102.0 meters
  2. Landing Pad 2 should be at [50,-73], serving 918 locations, route is 4126.4 meters

3) If you use 3 drone(s), the total route will be 8335.8 meters
  1. Landing Pad 1 should be at [51,-4], serving 799 locations, route is 2803.2 meters
  2. Landing Pad 2 should be at [49,-117], serving 469 locations, route is 2581.3 meters
  3. Landing Pad 3 should be at [49,115], serving 536 locations, route is 2951.3 meters

4) If you use 4 drone(s), the total route will be 8387.0 meters
  1. Landing Pad 1 should be at [50,-43], serving 611 locations, route is 2220.3 meters
  2. Landing Pad 2 should be at [50,147], serving 306 locations, route is 2119.0 meters
  3. Landing Pad 3 should be at [51,-142], serving 291 locations, route is 1931.5 meters
  4. Landing Pad 4 should be at [49,44], serving 598 locations, route is 2116.3 meters

Select number of drones (1-4): 1
  Saved ../solutions/Almond9832_1_solution_8374.txt
==> Visualization saved as ../solutions/Almond9832_OVERALL_SOLUTION.jpeg

All done. Routes saved in /solutions/ folder.
(base) allenbiju@Allens-MacBook-Pro-2 src %
```

Figure 4: CLI Output for Case 1 – *Almond9832.txt*

Visualization of Best Route (Almond9832.txt) – 1 Drone:

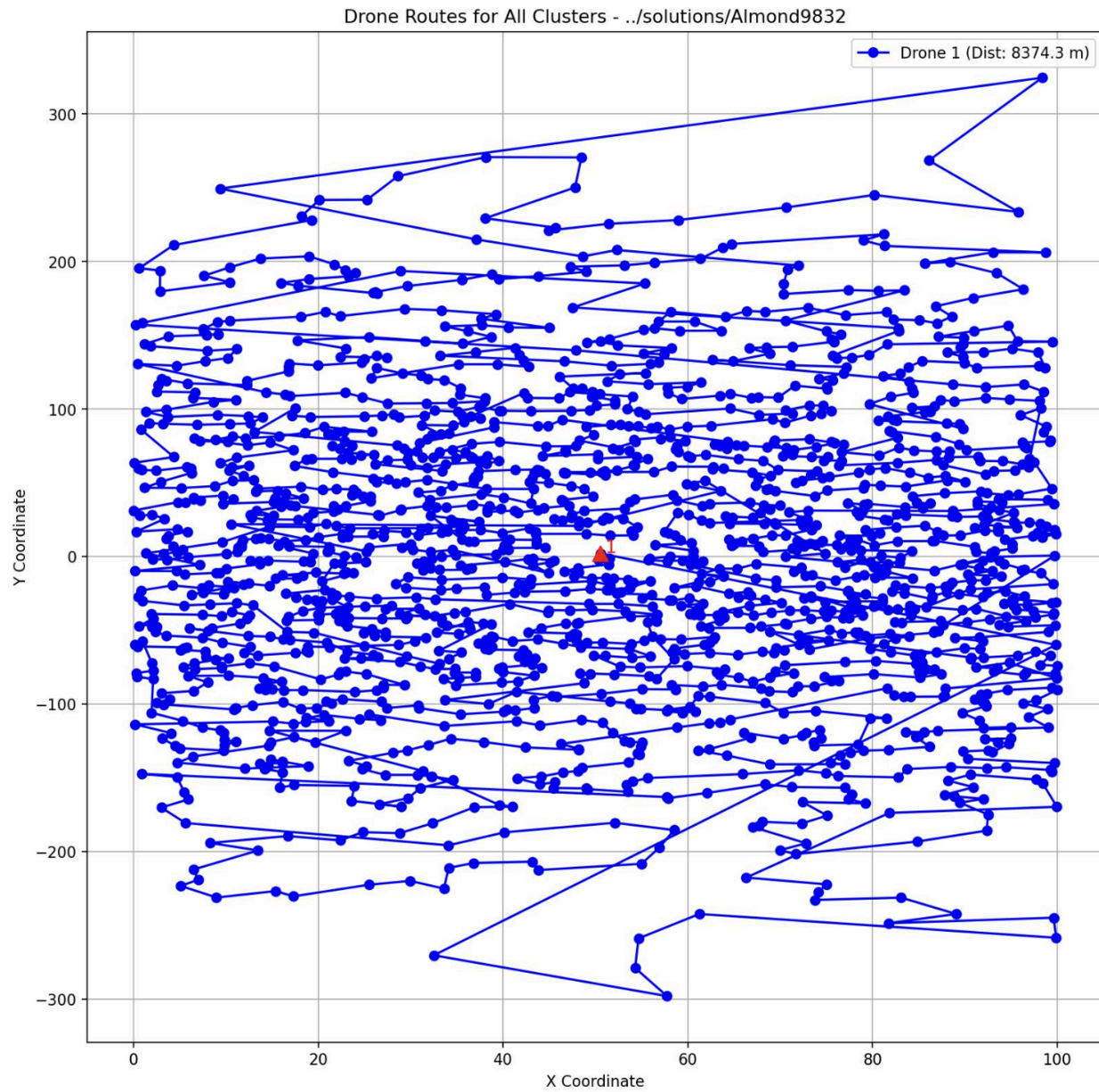


Figure 5: Best Route Visualization for *Almond9832.txt* ($K = 1$)

Visualization of Best Route (Almond9832.txt) – 2 Drones:

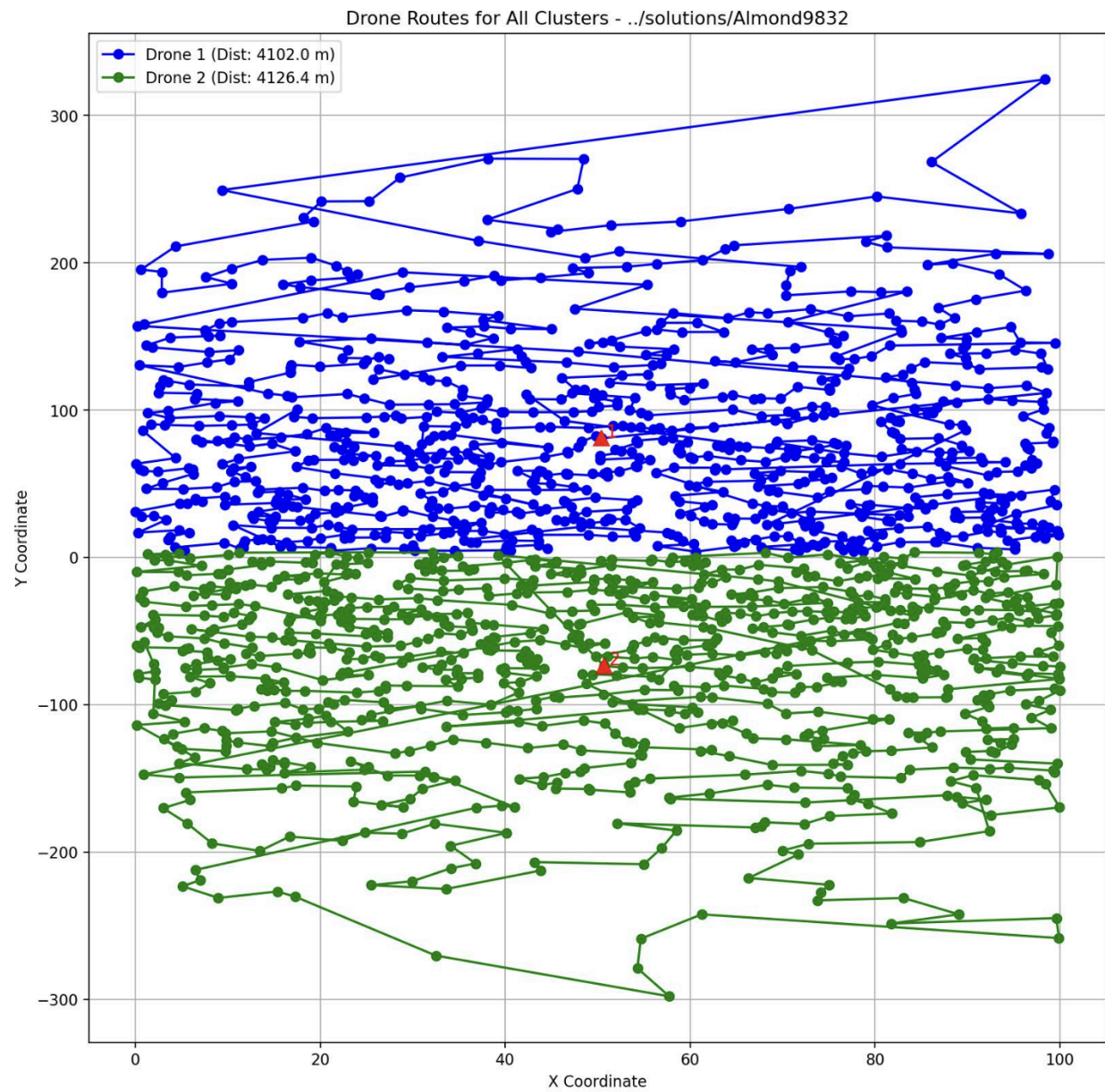


Figure 6: Best Route Visualization for *Almond9832.txt* ($K = 2$)

Visualization of Best Route (Almond9832.txt) – 3 Drones:

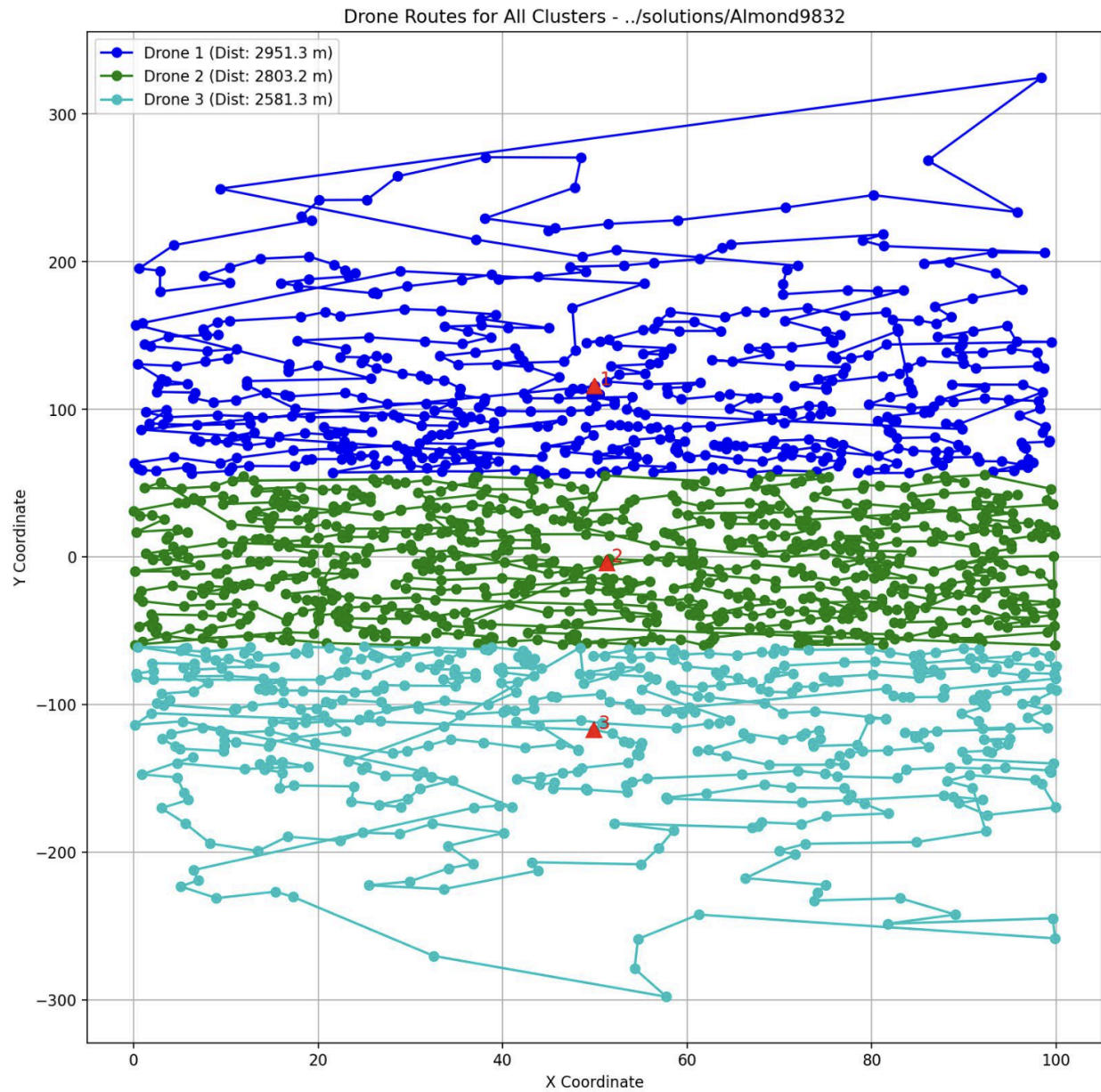


Figure 7: Best Route Visualization for *Almond9832.txt* ($K = 3$)

Visualization of Best Route (Almond9832.txt) – 4 Drones:

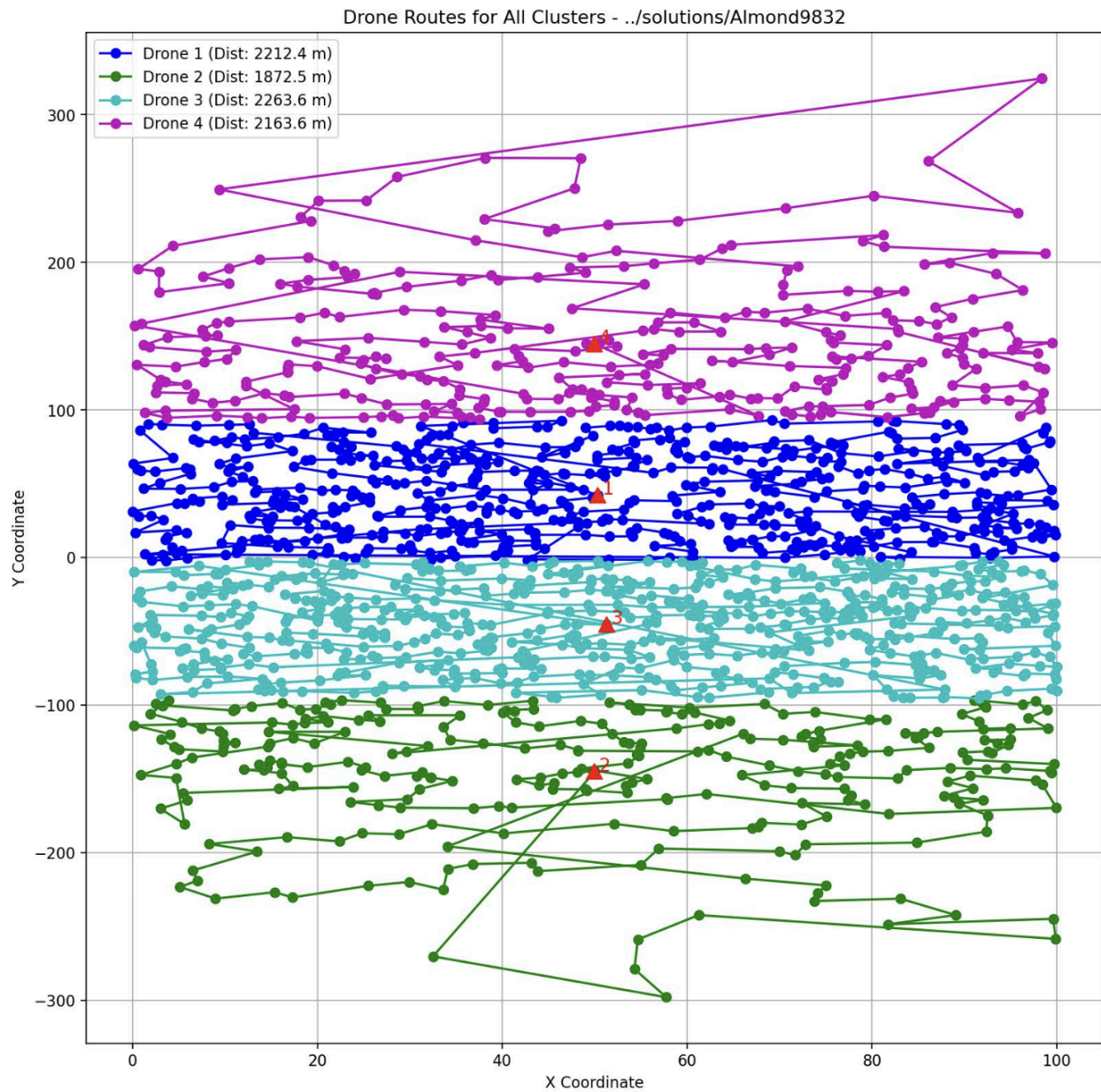


Figure 8: Best Route Visualization for *Almond9832.txt* ($K = 4$)

Case 2 – Pecan1212.txt

→ Inputs: *pecan1212.txt*, 1

```
● (base) allenbiju@Allens-MacBook-Pro-2 src % python main.py
Enter the name of file: pecan1212.txt
There are 1473 nodes: Solutions will be available by 10:13pm

1) If you use 1 drone(s), the total route will be 1834.8 meters
  1. Landing Pad 1 should be at [53,55], serving 1475 locations, route is 1834.8 meters

2) If you use 2 drone(s), the total route will be 1796.1 meters
  1. Landing Pad 1 should be at [25,69], serving 474 locations, route is 419.5 meters
  2. Landing Pad 2 should be at [67,48], serving 1003 locations, route is 1376.6 meters

3) If you use 3 drone(s), the total route will be 1767.4 meters
  1. Landing Pad 1 should be at [79,67], serving 496 locations, route is 687.6 meters
  2. Landing Pad 2 should be at [25,69], serving 474 locations, route is 419.5 meters
  3. Landing Pad 3 should be at [55,30], serving 509 locations, route is 660.3 meters

4) If you use 4 drone(s), the total route will be 1719.9 meters
  1. Landing Pad 1 should be at [55,36], serving 247 locations, route is 321.9 meters
  2. Landing Pad 2 should be at [79,67], serving 493 locations, route is 646.2 meters
  3. Landing Pad 3 should be at [54,25], serving 267 locations, route is 332.3 meters
  4. Landing Pad 4 should be at [25,69], serving 474 locations, route is 419.5 meters

Select number of drones (1-4): 1
Saved ../solutions/pecan1212_1_solution_1834.txt
==> Visualization saved as ../solutions/pecan1212_OVERALL_SOLUTION.jpeg

All done. Routes saved in /solutions/ folder.
(base) allenbiju@Allens-MacBook-Pro-2 src %
```

Figure 9: CLI Output for Case 2 – *pecan1212.txt*

Visualization of Best Route (pecan1212.txt) – 1 Drone:

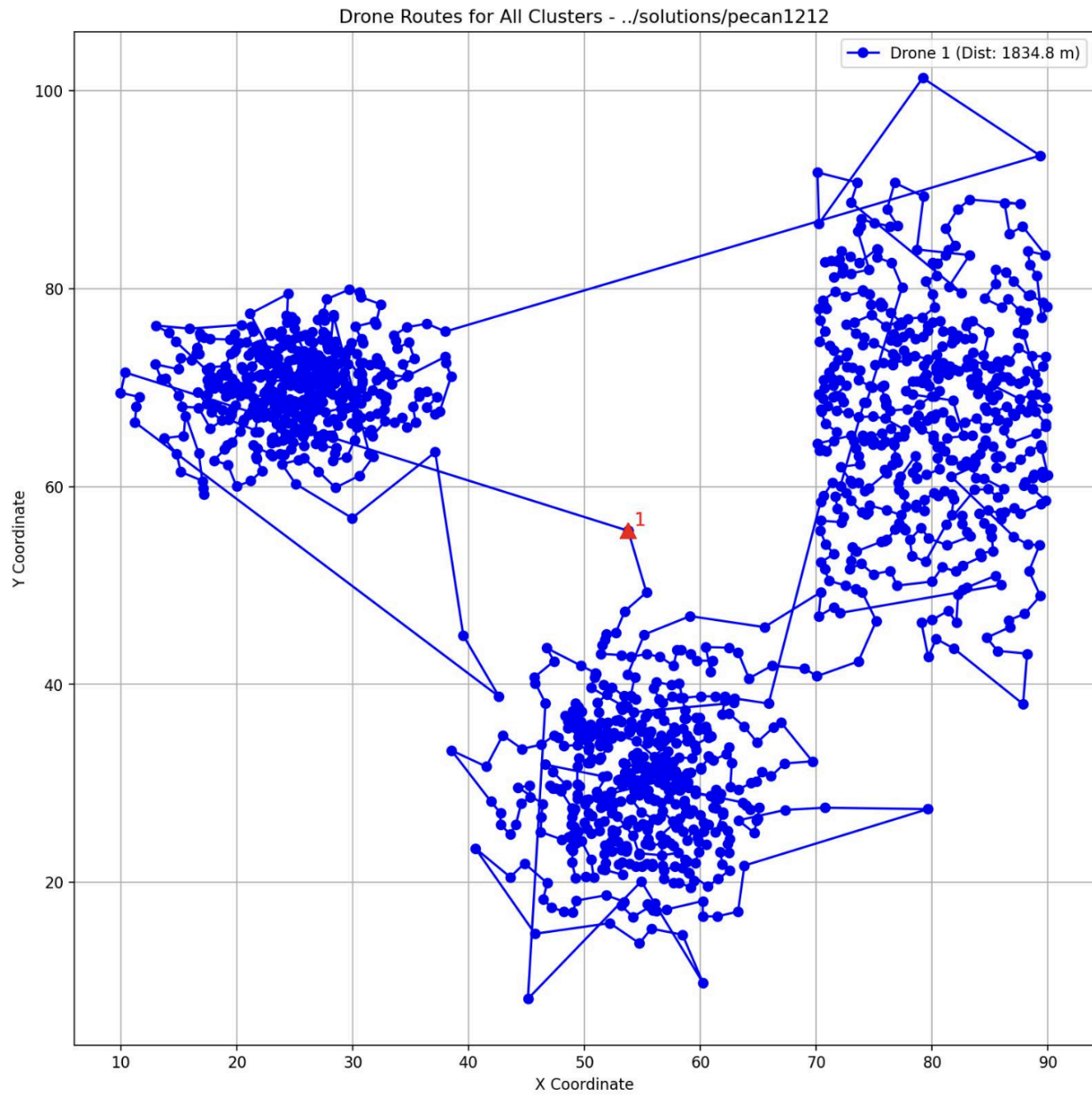


Figure 10: Best Route Visualization for *pecan1212.txt* ($K = 1$)

Visualization of Best Route (pecan1212.txt) – 2 Drones:

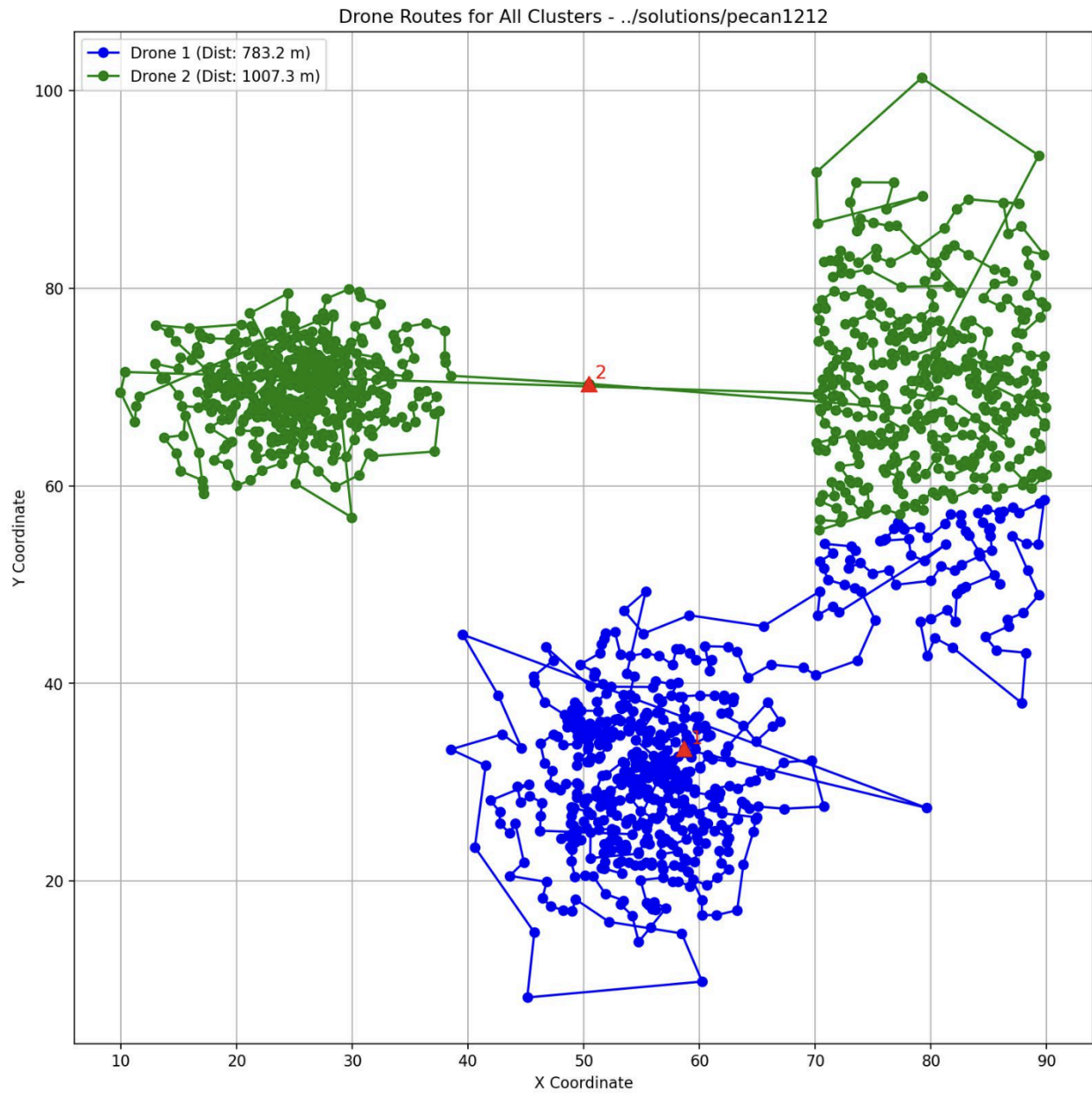


Figure 11: Best Route Visualization for *pecan1212.txt* ($K = 2$)

Visualization of Best Route (pecan1212.txt) – 3 Drones:

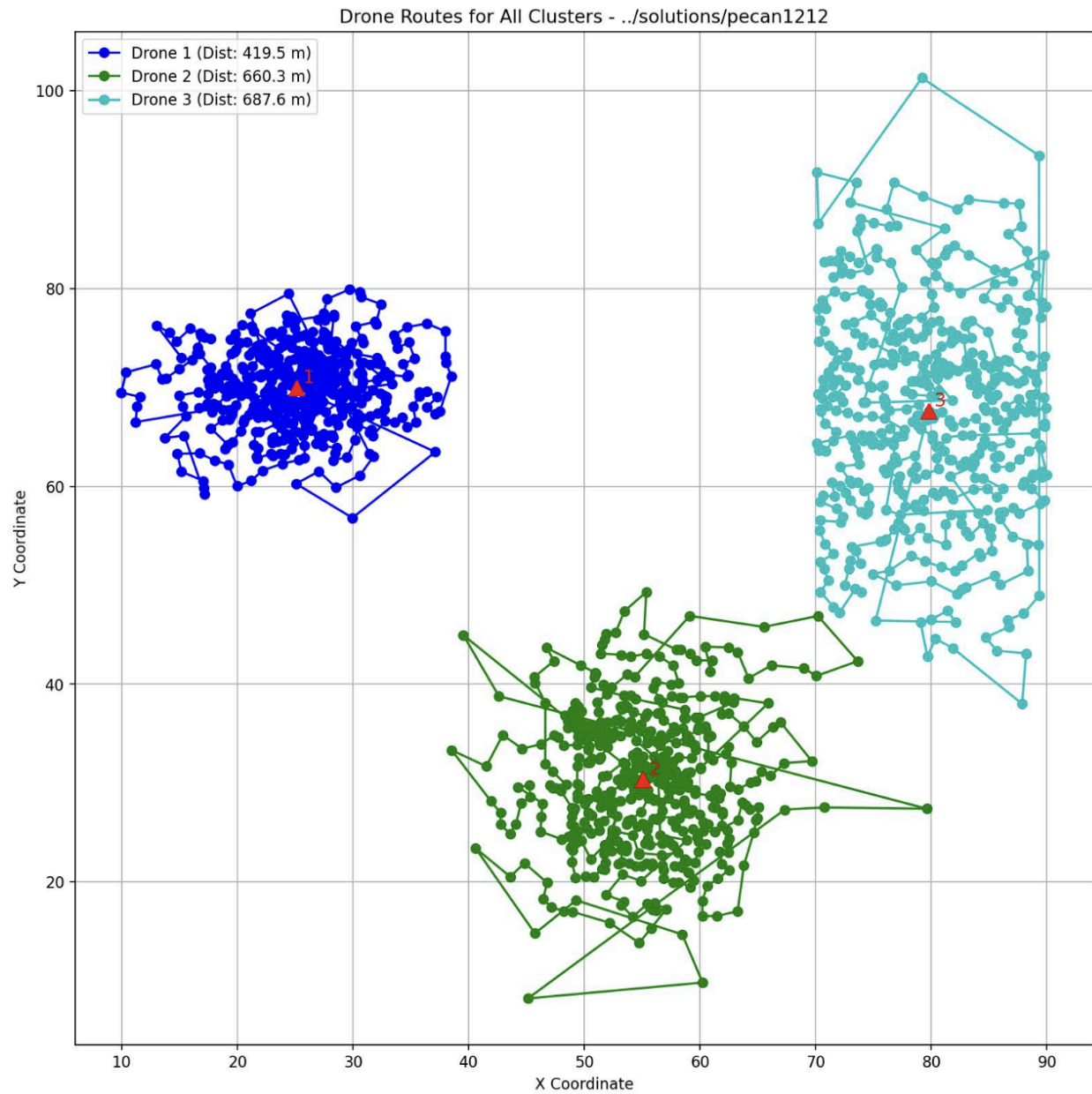


Figure 12: Best Route Visualization for *pecan1212.txt* ($K = 3$)

Visualization of Best Route (pecan1212.txt) – 4 Drones:

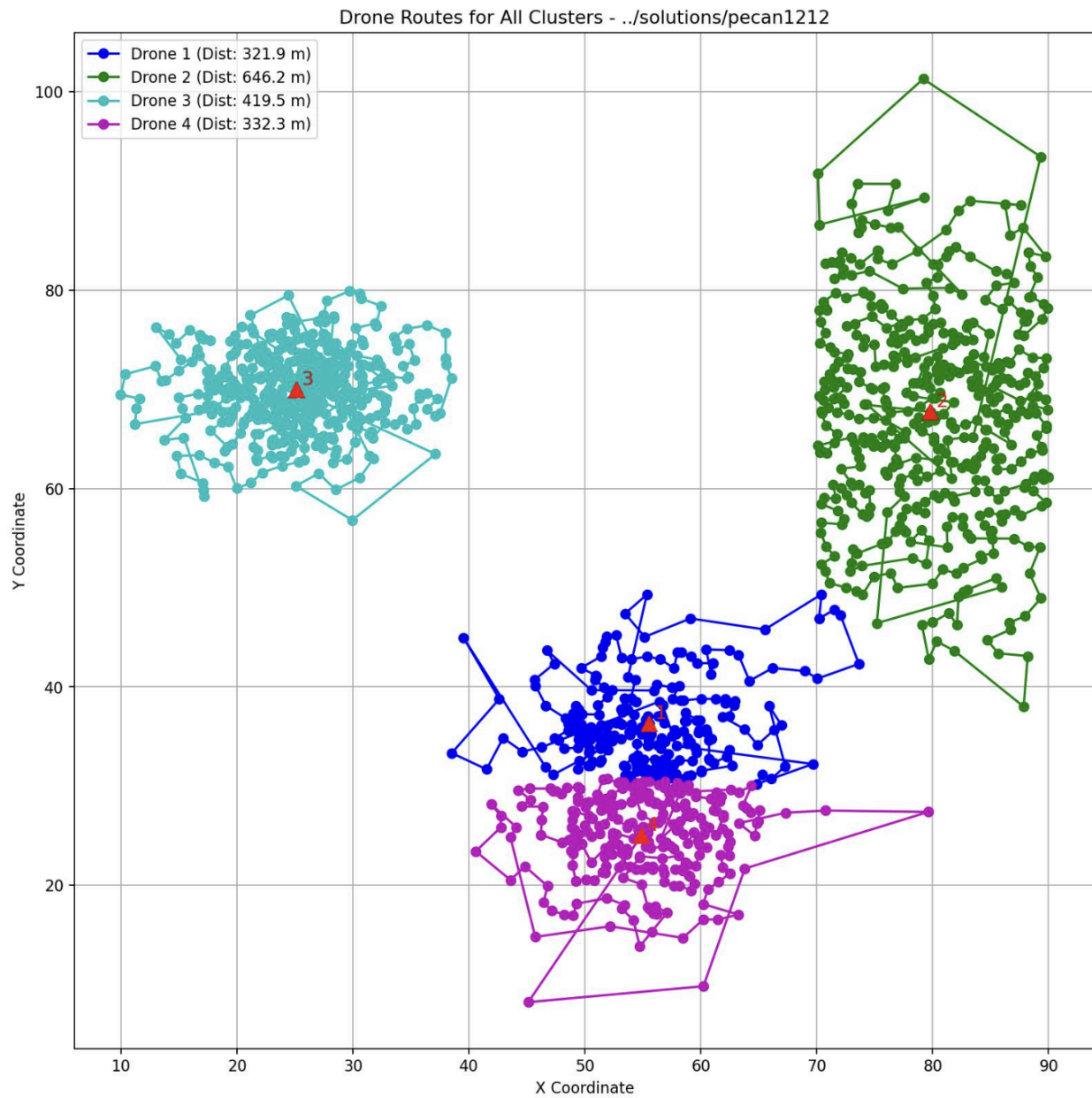


Figure 13: Best Route Visualization for *pecan1212.txt* ($K = 4$)

Case 3 – Walnut2621.txt

→ Inputs: *Walnut2621.txt*, 1

```
● (base) allenbiju@Allens-MacBook-Pro-2 src % python main.py
Enter the name of file: Walnut2621.txt
There are 994 nodes: Solutions will be available by 10:14pm

1) If you use 1 drone(s), the total route will be 2136.3 meters
  1. Landing Pad 1 should be at [39,42], serving 996 locations, route is 2136.3 meters

2) If you use 2 drone(s), the total route will be 2120.8 meters
  1. Landing Pad 1 should be at [18,46], serving 454 locations, route is 1207.7 meters
  2. Landing Pad 2 should be at [57,38], serving 544 locations, route is 913.1 meters

3) If you use 3 drone(s), the total route will be 2124.6 meters
  1. Landing Pad 1 should be at [20,69], serving 232 locations, route is 635.4 meters
  2. Landing Pad 2 should be at [58,40], serving 509 locations, route is 814.2 meters
  3. Landing Pad 3 should be at [18,22], serving 259 locations, route is 675.0 meters

4) If you use 4 drone(s), the total route will be 2156.1 meters
  1. Landing Pad 1 should be at [18,22], serving 260 locations, route is 675.9 meters
  2. Landing Pad 2 should be at [19,69], serving 224 locations, route is 586.0 meters
  3. Landing Pad 3 should be at [58,21], serving 241 locations, route is 439.7 meters
  4. Landing Pad 4 should be at [58,56], serving 277 locations, route is 454.4 meters

Select number of drones (1-4): 1
Saved ../solutions/Walnut2621_1_solution_2136.txt
==> Visualization saved as ../solutions/Walnut2621_OVERALL_SOLUTION.jpeg

All done. Routes saved in /solutions/ folder.
❏ (base) allenbiju@Allens-MacBook-Pro-2 src %
```

Figure 14: CLI Output for Case 3 – *Walnut2621.txt*

Visualization of Best Route (Walnut2621.txt) – 1 Drone:

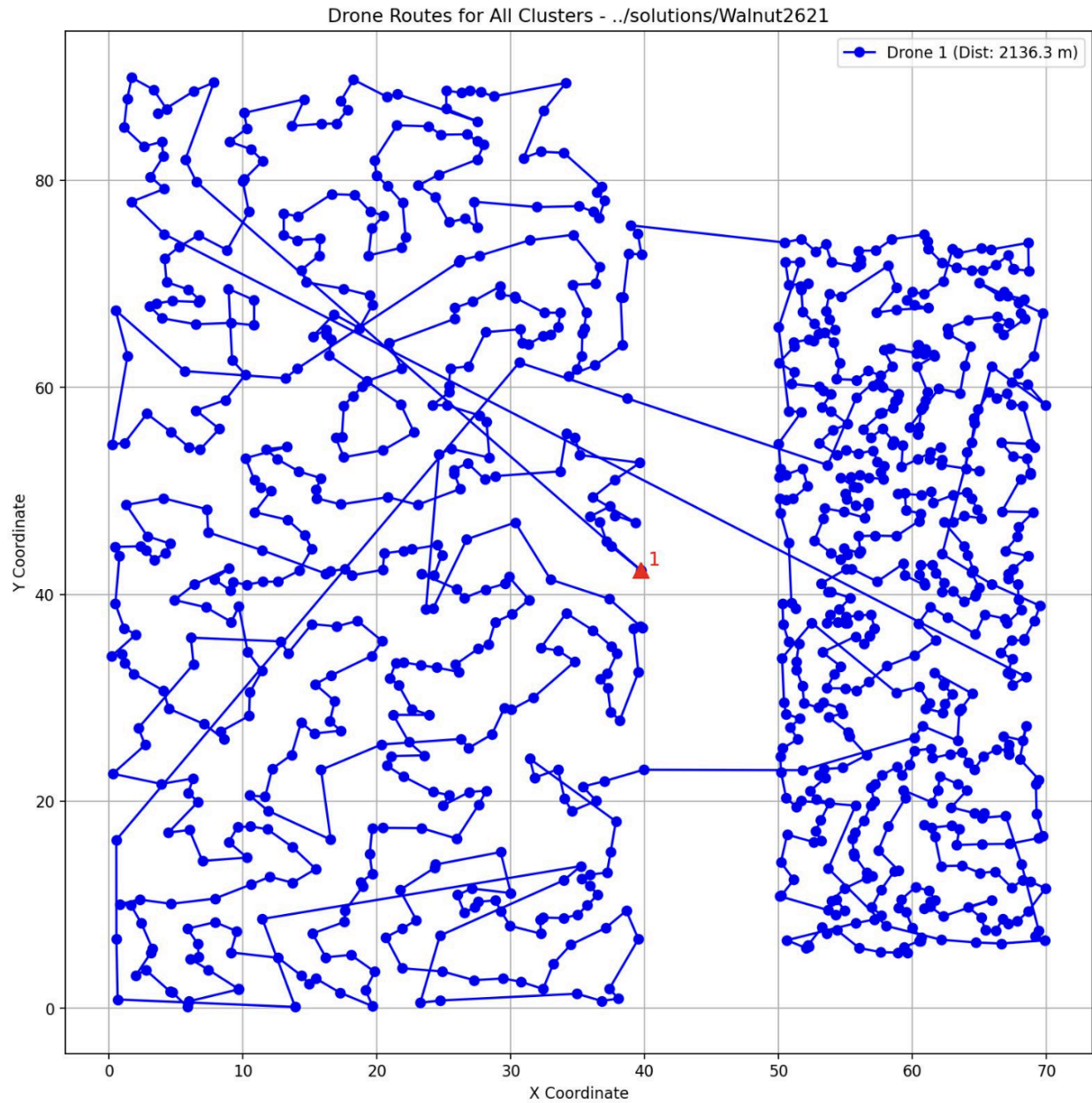


Figure 15: Best Route Visualization for *Walnut2621.txt* ($K = 1$)

Visualization of Best Route (Walnut2621.txt) – 2 Drones:

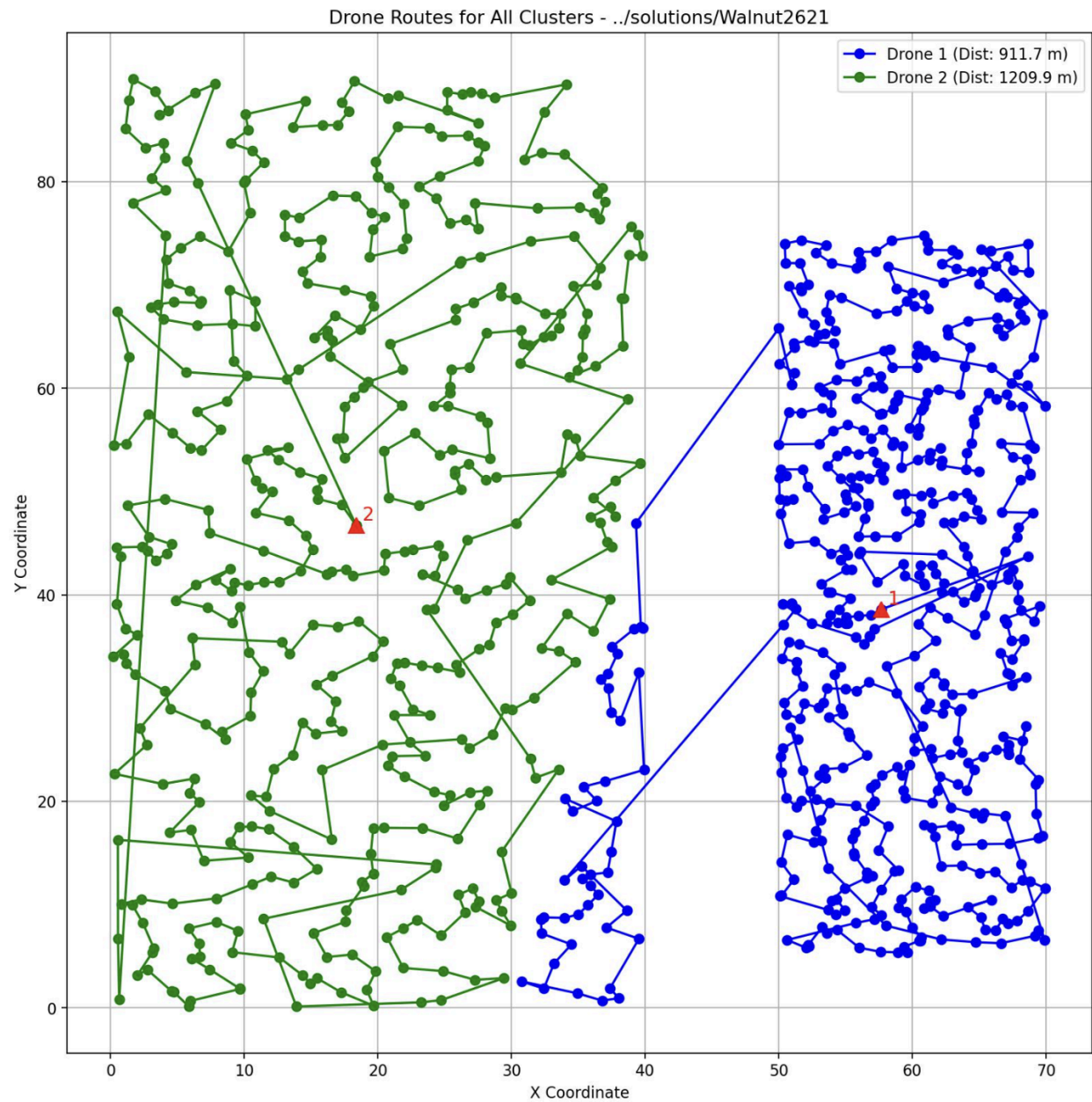


Figure 16: Best Route Visualization for *Walnut2621.txt* ($K = 2$)

Visualization of Best Route (Walnut2621.txt) – 3 Drones:

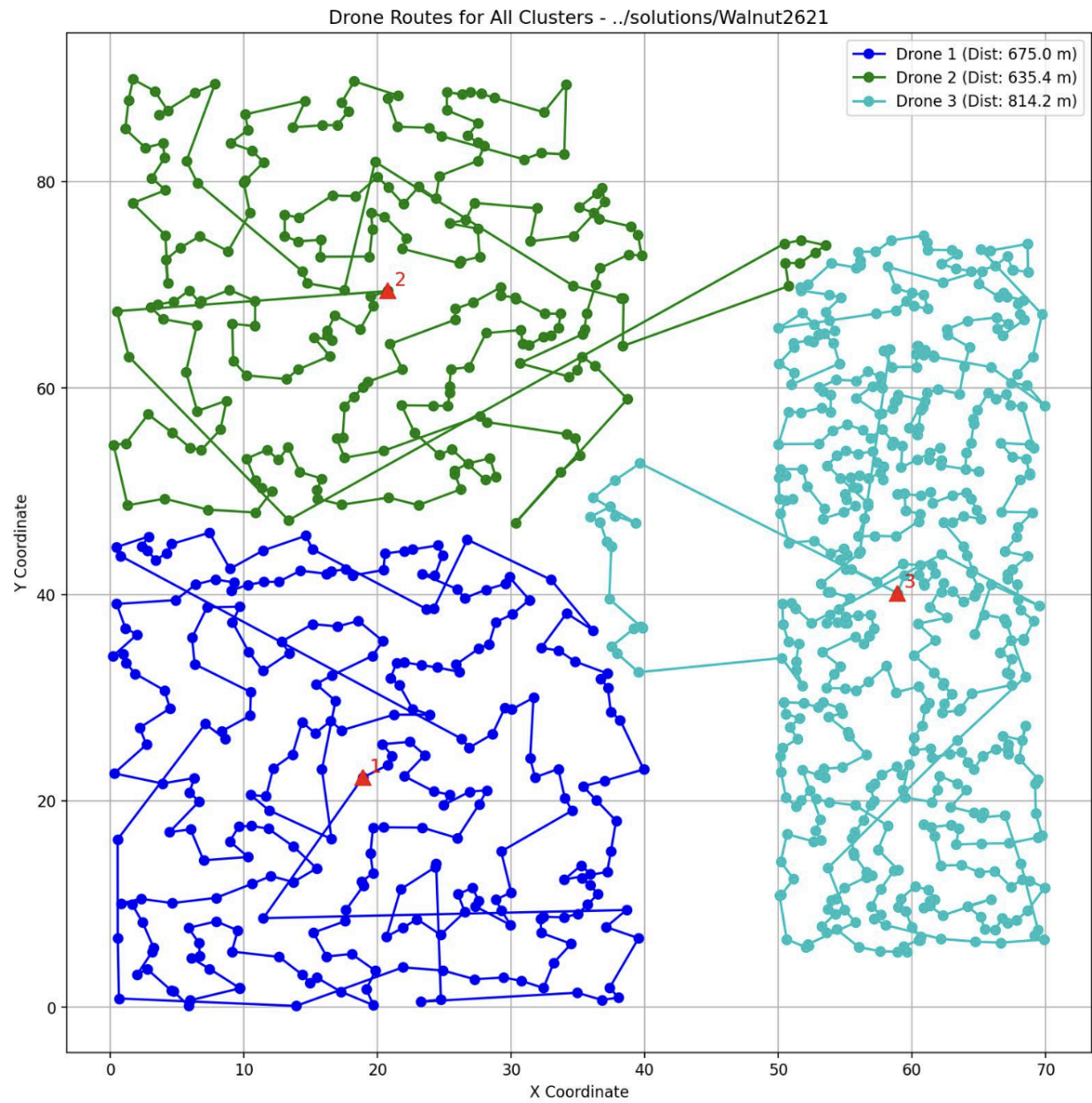


Figure 17: Best Route Visualization for *Walnut2621.txt* ($K = 3$)

Visualization of Best Route (Walnut2621.txt) – 4 Drones:

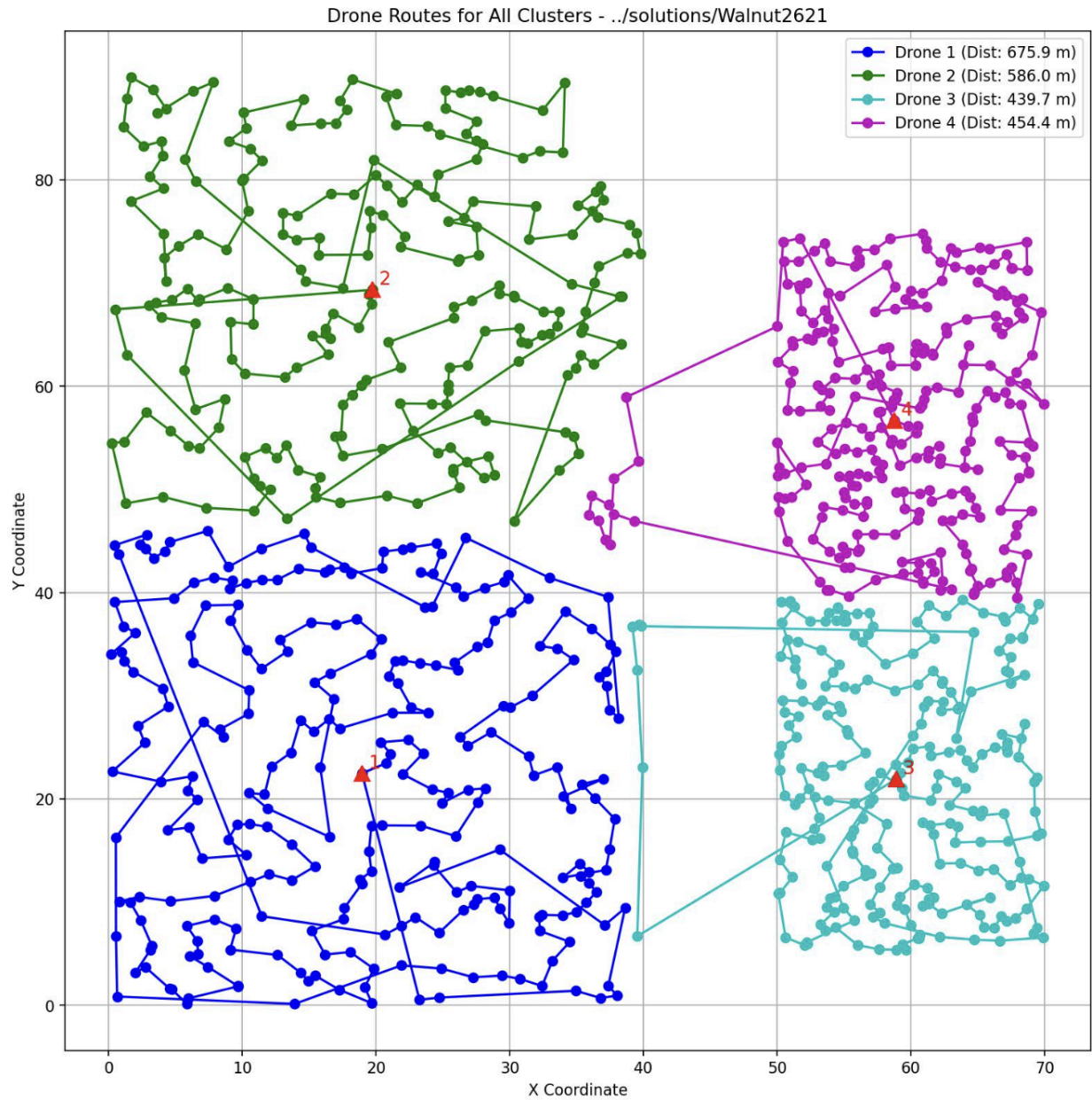


Figure 18: Best Route Visualization for *Walnut2621.txt* ($K = 4$)

Testing

7.1 Testing for Correct Program Behavior

To handle incorrect input and user errors, we added specific functionality to ensure our program could handle any issues and work as expected.

Case 1 – Missing File

→ **Condition:** The user provides a file name that does not exist in the data directory.

→ **Expected Behavior:** The program prints a message indicating that the file cannot be found and exits without crashing.

```
if not os.path.exists(filepath):  
    print(f"File not found: {filepath}")  
    return
```

Case 2 – Invalid File Format

→ **Condition:** The input file contains unreadable or non-numeric values instead of coordinate data.

→ **Expected Behavior:** The program should catch the error, display a warning message, and stop without running the clustering algorithm

```
coords = np.loadtxt(filepath)
```

Case 3 – Total Distance Exceeds 6000 Meters

→ **Condition:** The total calculated route distance exceeds 6000 meters.

→ **Expected Behavior:** The program should issue a warning saying that the total distance exceeds the 6000-meter constraint.

```
if bestSolution['totalDistance'] > 6000:  
    print(f"Warning: Solution is  
{bestSolution['totalDistance']:.1f}, which is greater than the 6000-meter  
constraint. ")
```

Case 4 – Dataset Too Large

→ **Condition:** The total number of coordinate nodes exceeds 4096.

→ **Expected Behavior:** The program should issue a warning saying that the number of nodes exceeds the 4096-node constraint.

```
if numNodes > 4096:  
    print(f"Warning: There are more than 4096 nodes.")
```

7.2 Quality of Solution

To verify the correctness and behavior of our implementation, we designed a series of controlled tests. Since our K-means algorithm is approximate, we cannot guarantee an optimal solution in every case, but we can check whether its output is reasonable under known conditions [3].

We generated datasets where the expected cluster structure is known:

1. **Single Circle Test** - We created a dataset of points evenly distributed along the perimeter of a unit circle centered near the origin (0, 0). This allowed us to confirm that the algorithm correctly identifies a cluster centered near the origin and that the visualization accurately shows the circular pattern of the data with $K = 1$. Figures 19 and 20 show the visualization and trace for this test dataset, called *circlePoints.txt*.

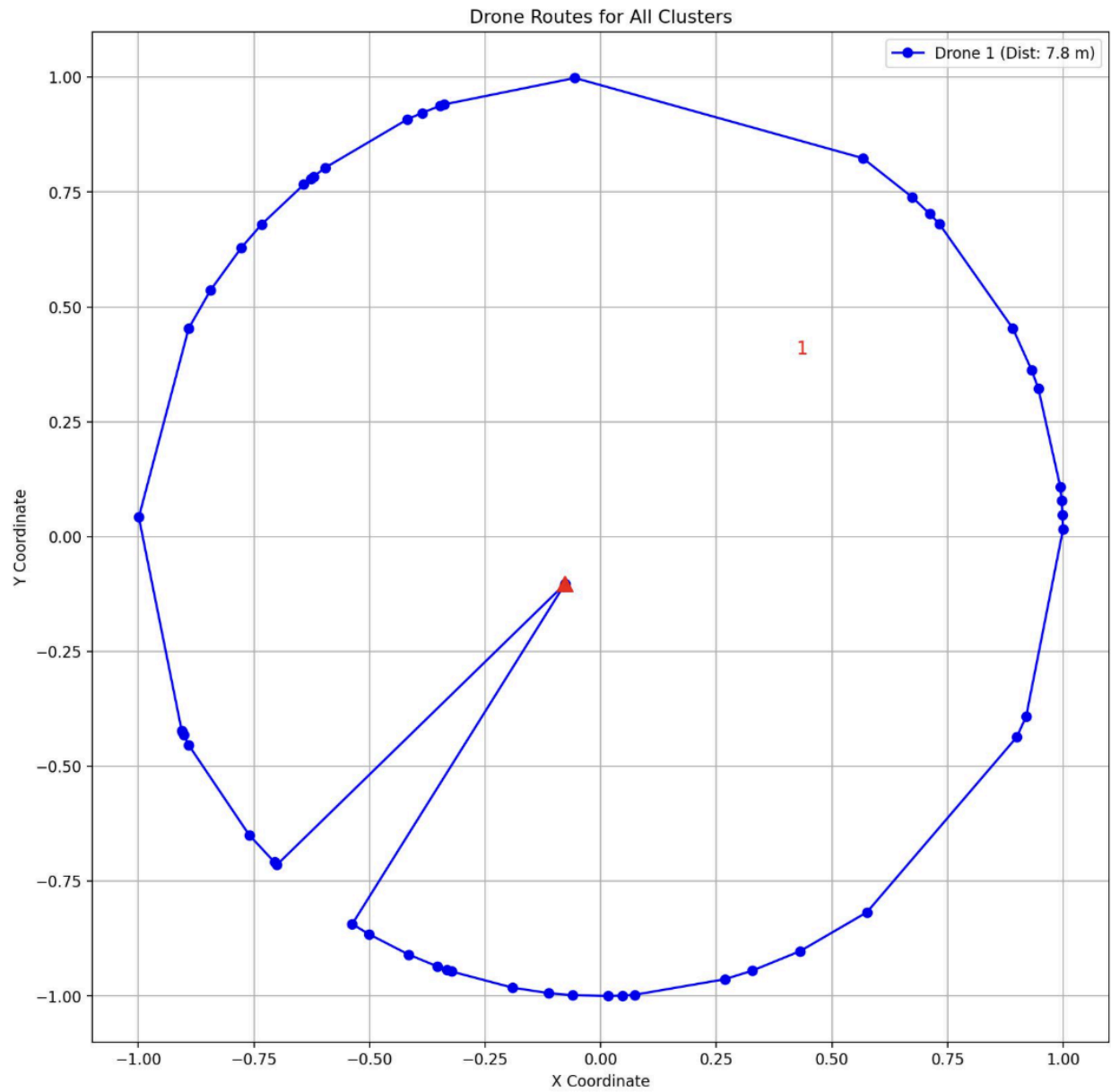


Figure 19: Circular Visualization of Points with $K = 1$ Cluster

```

(base) allenbiju@Allens-MacBook-Pro-2 src % python main.py
Enter the name of file: circlePoints.txt
There are 50 nodes: Solutions will be available by 10:16pm

1) If you use 1 drone(s), the total route will be 7.8 meters
  1. Landing Pad 1 should be at [0,0], serving 52 locations, route is 7.8 meters

2) If you use 2 drone(s), the total route will be 10.2 meters
  1. Landing Pad 1 should be at [0,0], serving 26 locations, route is 4.4 meters
  2. Landing Pad 2 should be at [0,0], serving 28 locations, route is 5.7 meters

3) If you use 3 drone(s), the total route will be 9.4 meters
  1. Landing Pad 1 should be at [0,0], serving 15 locations, route is 2.8 meters
  2. Landing Pad 2 should be at [0,0], serving 16 locations, route is 3.1 meters
  3. Landing Pad 3 should be at [0,0], serving 25 locations, route is 3.5 meters

4) If you use 4 drone(s), the total route will be 9.1 meters
  1. Landing Pad 1 should be at [0,0], serving 16 locations, route is 3.1 meters
  2. Landing Pad 2 should be at [0,0], serving 12 locations, route is 1.6 meters
  3. Landing Pad 3 should be at [0,0], serving 15 locations, route is 1.6 meters
  4. Landing Pad 4 should be at [0,0], serving 15 locations, route is 2.8 meters

Select number of drones (1-4): 1
Saved ../solutions/circlePoints_1_solution_7.txt
==> Visualization saved as ../solutions/circlePoints_OVERALL_SOLUTION.jpeg

All done. Routes saved in /solutions/ folder.
(base) allenbiju@Allens-MacBook-Pro-2 src %

```

Figure 20: Cluster Assignment and Trace for Circle Points

2. **Two-Circle Test** – We generated two circular clusters, one centered at (0, 0) and another at (0, 5), then ran our algorithm with $K = 2$. The output visualization showed two distinct clusters, with centers located near the expected coordinates, and displayed two well-separated circular patterns. Figures 21 and 22 show the visualization and trace for this test dataset, which we called *twoCirclesPoints.txt*.

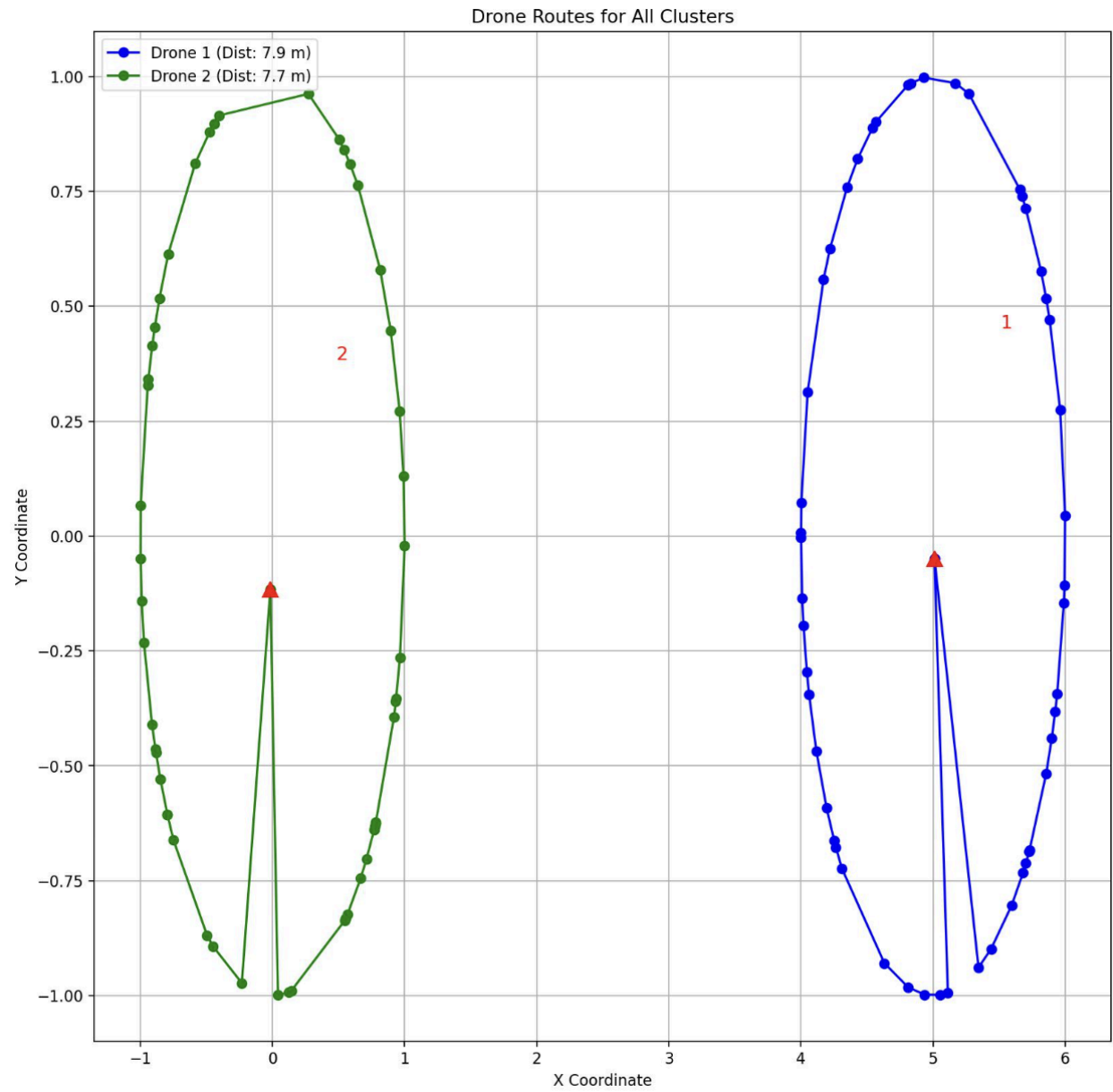


Figure 21: Two Distinct Circular Cluster Visualizations with $K = 2$

```

● (base) allenbiju@Allens-MacBook-Pro-2 src % python main.py
Enter the name of file: twoCirclesPoints.txt
There are 100 nodes: Solutions will be available by 10:19pm

1) If you use 1 drone(s), the total route will be 18.3 meters
  1. Landing Pad 1 should be at [2,0], serving 102 locations, route is 18.3 meters

2) If you use 2 drone(s), the total route will be 15.7 meters
  1. Landing Pad 1 should be at [5,0], serving 52 locations, route is 7.9 meters
  2. Landing Pad 2 should be at [0,0], serving 52 locations, route is 7.7 meters

3) If you use 3 drone(s), the total route will be 18.3 meters
  1. Landing Pad 1 should be at [5,0], serving 52 locations, route is 7.9 meters
  2. Landing Pad 2 should be at [0,0], serving 26 locations, route is 4.9 meters
  3. Landing Pad 3 should be at [0,0], serving 28 locations, route is 5.5 meters

4) If you use 4 drone(s), the total route will be 17.9 meters
  1. Landing Pad 1 should be at [0,0], serving 21 locations, route is 3.6 meters
  2. Landing Pad 2 should be at [0,0], serving 23 locations, route is 3.7 meters
  3. Landing Pad 3 should be at [0,0], serving 12 locations, route is 2.6 meters
  4. Landing Pad 4 should be at [5,0], serving 52 locations, route is 7.9 meters

Select number of drones (1-4): 2
Saved ../solutions/twoCirclesPoints_1_solution_7.txt
Saved ../solutions/twoCirclesPoints_2_solution_7.txt
==> Visualization saved as ../solutions/twoCirclesPoints_OVERALL_SOLUTION.jpeg

All done. Routes saved in /solutions/ folder.
(base) allenbiju@Allens-MacBook-Pro-2 src %

```

Figure 22: Cluster Assignment and Trace for Two Circles

3. **4096 Random Coordinate Test** – We generated a dataset of 4096 random nodes, called *randomCoordinates4096.txt*, to evaluate the upper bound of our processing time. This represents the maximum input size used to measure the performance of both the K-means procedure and the Nearest Neighbor method [5]. The entire run took only 75.2 seconds, which reflects the worst-case scenario for our current implementation.

These tests demonstrate that both our K-means clustering and visualization functions are working as intended. The cluster centers align with the data's actual structure, so we knew exactly where they were supposed to be, and the plotted routes confirm that our implementation produces geometrically consistent results.

7.3 Algorithm Performance

The K-means algorithm was used to group delivery points into optimal clusters for efficient pheromone delivery. Essentially, it iteratively assigns each point to the nearest landing pad and recalculates the landing positions until convergence. In our implementation, the algorithm was enhanced by integrating a

Nearest Neighbor (NN) heuristic to determine the most efficient route for each cluster. The NN algorithm ensures that the drones follow a minimal distance path that starts and ends at the cluster launch pad [7].

The algorithm's performance was evaluated across three datasets: *Almond9832.txt*, *pecan1212.txt*, and *Walnut2621.txt*. For each cluster, values $K = 1$ to $K = 4$. The Objective Function (OF), which measures the total within-cluster variance, consistently decreases as the number of clusters increases. For example, the OF dropped from 18100580.2 to 3304173.3 for the *Almond9832.txt* dataset, from 1343386.2 to 112974.9 for the *pecan1212.txt* dataset, and from 1025940.8 to 217991.9 for the *Walnut2621.txt* dataset. This reduction indicates that the K-means effectively minimized intra-cluster distances as the cluster became more refined.

Objective Function

As requested by Dr. Keogh, Table 2 presents the objective function (OF) scores obtained from our K-means algorithm on three calibration datasets. Although we do not have a reference “correct” value to compare against, we observe that the OF generally decreases as the number of clusters, K , increases, which aligns with expectations. This trend suggests that the algorithm is effectively reducing within-cluster variance as more clusters are added.

Table 2: The Objective Function for Increasing K on Three Datasets

	OF when $K = 1$	OF when $K = 2$	OF when $K = 3$	OF when $K = 4$
<i>Almond9832.txt</i>	18100580.2	7368133.0	4541097.2	3304173.3
<i>pecan1212.txt</i>	1343386.2	836193.8	128880.8	112974.9
<i>Walnut2621.txt</i>	1025940.8	618464.8	370091.3	217991.9

To improve visualization, we generated elbow plots to identify the optimal number of clusters for each dataset. The results of our final algorithm showed that *Almond9832.txt*, *pecan1212.txt*, and *Walnut2621.txt* reached their optimal points at 3, 2, and 3 clusters, respectively. This indicates that the Almond and Walnut required more clusters due to their wider data spread, whereas the pecan was more compact, requiring only two. Overall, the elbow plot results help further confirm that the K-means algorithm effectively determined the most efficient cluster structure for each dataset.

Elbow Plots

This graph shows the sum of squared errors of each cluster trace with different K values (1-4). The y-axis represents the output of the objective function, which was implemented below in Figures 23 through 25, and wherever the graph “elbows” is the optimal number of clusters for this specific dataset.

Looking at the graph in Figure 23, there is an apparent “elbow” at $K = 3$, which corresponds to the trace results from our algorithm for *pecan1212.txt*, as seen above in Figure 9. The least-cost trace for Mr. Keogh is the one that uses three drones.

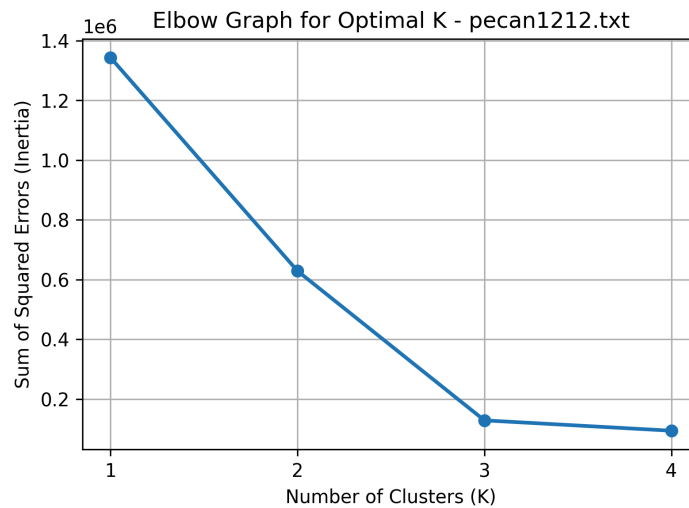


Figure 23: Elbow Graph for *pecan1212.txt*

Figure 24 shows the elbow pattern for the *Almond9832.txt* dataset. The error drops sharply from $K = 1$ to $K = 2$ and then levels off, indicating that the elbow occurs around $K = 2$. This aligns with the routing results for this dataset, where the 2-drone configuration produces the lowest total distance.

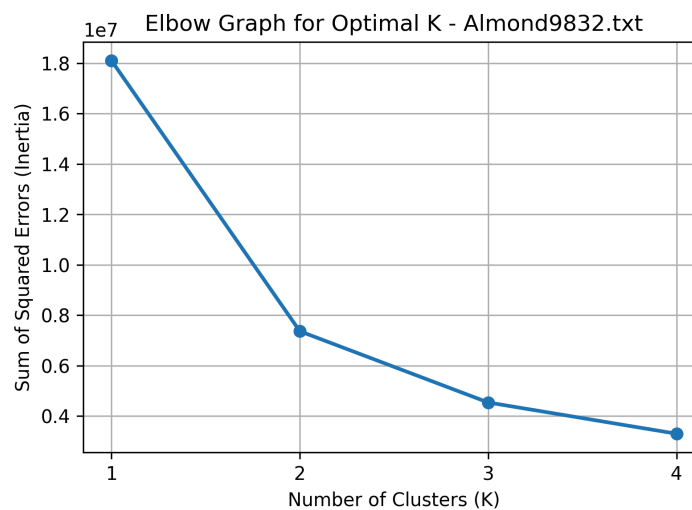


Figure 24: Elbow Graph for *Almond9832.txt*

Figure 25 illustrates the elbow curve for the *Walnut2621.txt* dataset. The steep decline from $K = 1$ to $K = 2$, followed by more minor improvements through $K = 3$ and $K = 4$, suggests the elbow lies between $K = 2$ and $K = 3$. Consistent with this, the 3-drone solution yields the best overall cost for this case.

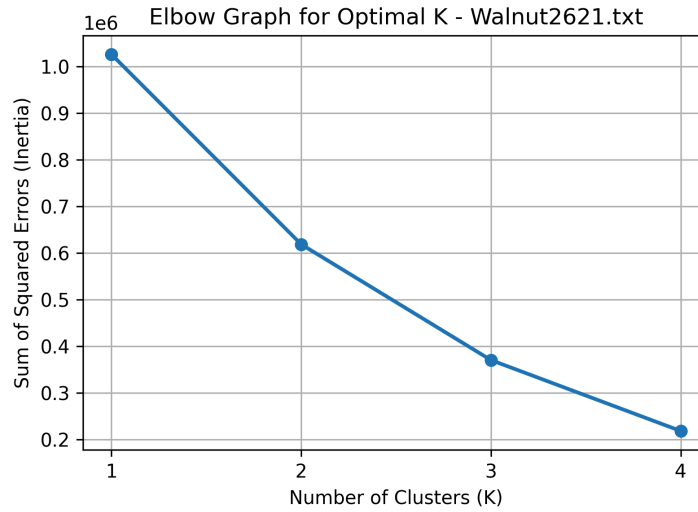


Figure 25: Elbow Graph for *Walnut2621.txt*

Final Test

To validate the algorithm's performance under more realistic conditions, additional tests were run on the K-means algorithm using the three datasets: *pecan1212.txt*, *Almond9832.txt*, and *Walnut2621.txt*. We ran the program 5 times to obtain the average total route distance for 1-4 drones, as shown in Table 3.

Table 3: The average total distances of each set of drones using the K-means algorithm on three datasets

Dataset	Average Total Route Distance of 1 Drone(s)	Average Total Route Distance of 2 Drone(s)	Average Total Route Distance of 3 Drone(s)	Average Total Route Distance of 4 Drone(s)	Runtime
<i>pecan1212.txt</i>	1834.8 m	1791.6 m	1767.4 m	1719.8 m	10.2 s
<i>Almond9832.txt</i>	8374.3 m	8228.4 m	8335.8 m	8469.4 m	14.8 s
<i>Walnut2621.txt</i>	2136.3 m	2109.9 m	2122.5 m	2156.1 m	4.5 s

Conclusion

In this project, we developed an efficient routing system that helps Mr. Keogh deploy pheromone-delivery drones across large orchards. By using K-means clustering to determine drone regions and Nearest Neighbor routing to plan each drone's path, the system generates delivery plans for up to four drones.

To determine the number of drones that would most efficiently visit all the stations, we calculated the objective function. As the number of clusters increased, we found that the OF values decreased. This insight shows that with more clusters, the drones can cover all the required stations more efficiently.

References

- [1] Keogh, P. (n.d.). *P2 Detailed Briefing* [PowerPoint slides].
- [2] Brownlee, J. (n.d.). *Tutorial to implement k-nearest neighbors in Python from scratch*. Machine Learning Mastery.
<https://machinelearningmastery.com/tutorial-to-implement-k-nearest-neighbors-in-python-from-scratch/>
- [3] GeeksforGeeks. (n.d.). *K-nearest neighbors in machine learning*.
<https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>
- [4] Brownlee, J. (n.d.). *Hyperparameter optimization with random search and grid search*. Machine Learning Mastery.
<https://machinelearningmastery.com/hyperparameter-optimization-with-random-search-and-grid-search/>
- [5] W3Schools. (n.d.). *Python file open*. https://www.w3schools.com/python/python_file_open.asp
- [6] GeeksforGeeks. (n.d.). *Heuristic search techniques in artificial intelligence*.
<https://www.geeksforgeeks.org/artificial-intelligence/heuristic-search-techniques-in-ai/>
- [7] GeeksforGeeks. (n.d.). *Greedy best-first search algorithm*.
<https://www.geeksforgeeks.org/dsa/greedy-best-first-search-algorithm/>
- [8] GeeksforGeeks. (n.d.). *Approximate nearest neighbor (ANN) search in machine learning*.
<https://www.geeksforgeeks.org/machine-learning/approximate-nearest-neighbor-ann-search/>
- [9] W3Schools. (n.d.). *Matplotlib plotting in Python*.
https://www.w3schools.com/python/matplotlib_plotting.asp